The Brilliant Employee · 06

# Every backup died. Nothing logged it.

I cannot tell you how many nightly backup jobs were dead. Exit code 78 killed each one before it could write a line.

sgnk.ai/brilliant ↗

Sagnik Mitra

# Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

| THE SYSTEM, AS OF 7 AUGUST 2026, 22:55Z | |
|---|---|
| **rules written** | 11 June, 57 days ago |
| **ledger running** | 27 June, 41 days |
| **tasks in the ledger** | 3,140, across 3,680 rows |
| **controls on tool calls** | **6 registered, 4 that can refuse** |

**You can rent the model. You cannot rent the part that tells you it was wrong.**

# Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

| | |
|---|---|
| **sizing check** | sizes a job before a model is picked |
| **model picker** | recommends which model gets the job |
| **safety stops** | refuse the irreversible, before it runs |
| **grader** | grades finished work pass or fail |
| **task log** | at least one append-only line per task |

Listed in the order they act on a task, not the order they were built.

Today cuts across more than one, so no row is marked.

> **ASIDE.** Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

**This page is the map. Nothing here needs another post to make sense.**

# Every backup died and nothing said a word

Nothing crashed. Nothing complained. No red banner, no failed-run email, no log line asking for attention. Every scheduled backup job pointed at my project folders died the instant it started, night after night.

| HOW MANY JOBS DIED: THREE RECORDS, THREE COUNTS | |
|---|---|
| my rule file, typed that night | 16 |
| the plists I later archived | 15 |
| a source comment from the week between | 14 |

The one scheduled job that lived in a plain hidden folder ran perfectly the whole time. That is precisely why I never went looking. A green light in one corner of the room reads, wrongly, as a green room.

> **Plain words: EXIT 78.** the code a macOS background job returns when the operating system refuses it before it can do, or say, anything

---

**The absence of complaints is not the presence of backups.**

# The OS killed the jobs before they could log

- The scheduler fires the backup job on time, every night
  `macOS launchd`
- The job reaches into a project folder on the Desktop
  `the backup script`

  **refused: that path needs Full Disk Access**
- It dies with exit code 78 before it opens an output stream
  `no crash, no alert, no log line`

The refusal lands before the script's first line. It is documented, it is old ground, and it still took out the fleet.

> **ASIDE.** For engineers: scheduled launchd agents hitting TCC-protected paths fail with getcwd errors and exit 78 on chdir. Keep scheduled scripts and their data on plain paths like ~/.sgnk or ~/Library, or grant Full Disk Access to a dedicated runner.

The macOS trivia is not the lesson. The lesson is that a safety system failed in a way that produced zero evidence, and every green assumption I held survived contact with nothing.

---

**The interesting bug is never the mechanism. It is the silence around it.**

# One survivor made the whole fleet look alive

**16 by my note, into project folders**

stopped dead at the permission wall, exit 78, not one log line

**the invisible OS permission wall**

macOS guards Desktop, Documents and Downloads against background jobs

**1 job in a plain hidden folder**

no wall on its path, landed its output perfectly every night

The living job was my end-of-day archiver, parked under a dot-folder the wall does not guard. Its siblings a few path segments away hit the wall and vanished without a sound, and I cannot tell you how many there were. I saw the survivor and generalized.

> **ASIDE.** It is hard to overstate how convincing one working job is when you are deciding whether to audit the rest of the fleet.

---

**One job that works hides fifteen that do not, because both print nothing.**

# Dead and alive produce identical output

| THE DEAD JOBS | 1 LIVING JOB |
|---|---|
| crash<br>**none** | crash<br>**none** |
| alert<br>**none** | alert<br>**none** |
| log line<br>**none** | log line<br>**quiet success** |
| what I heard<br>**silence** | what I heard<br>**silence** |

From where I sat, the two columns are the same column. A backup that ran and had nothing to report makes exactly the sound of a backup the OS killed on arrival. Failure arrived in the same tone as success, and it kept arriving nightly until I finally checked the dumps themselves.

My ledger records two separate silent outages of this same backup fleet. The permission wall took out the whole scheduled set at once. Later, an unrelated one-line shell bug kept the nightly backups dead for weeks. Different killers, identical symptom: silence.

---

**If success and failure sound identical,
you are not monitoring, you are hoping.**

# A review queue went silent the same way

My automation system's finished work queues up for my human verdict. The count sat at 2 of a required 5 for days, and everyone, including me, read that as the human being slow.

| WHAT THE QUIET QUEUE WAS HIDING | |
|---|---|
| **reported** | 0 items awaiting review, no error |
| **human verdicts** | 2 of a required 5, for days |
| **rows in the pool** | 85 |
| **machine rederivations posing as gold** | **83 of 85** |
| the sampler retired every item carrying any machine-written row: it ate its own pool | |

Zero to review never meant the work was judged. It meant the pipeline had quietly made judging impossible, and it said so in the same tone as a job well done.

---

**Nothing-to-review and nothing-can-be-reviewed print the same zero.**

# A zero with no denominator is not a measurement

$$\frac{0}{?}$$

every quiet report is this fraction until something states the denominator

A checker that finds no problems and a checker that is broken emit the same message. Only a stated denominator separates them. Two of the rows below I had written up as built, and both fell over when I opened the script.

| DENOMINATORS: AS BUILT, AGAINST AS CLAIMED | |
|---|---|
| repos checked | printed by the probe |
| repos failing | printed by the probe |
| **how large each dump is** | **never printed** |
| **the sampler's pool size** | **computed, never printed** |

**Never accept a zero that does not say zero out of what.**

# Only a restored file is a backup

The schedule existing is an intention. The job firing is an intention. Only a restored file is a backup.

```
sgnk-backup-probe.sh:54, the denominator line

printf '%b' "[backup-probe]
  $bad failing /
  $(printf '%s' "$REPOS" | wc -w)
  probed\n$lines"
```

1. Every quiet job reports a denominator, not a feeling
2. A green light must be a positive signal, never the absence of a red one
3. Pull on the net on a schedule: open a dump, restore one table
4. Put scheduled work on paths the OS does not silently guard

This morning it used that honesty on me: two repos watched, two failing, the newest dump well past its threshold. The net is the only reason I know.

---

**A backup you have never restored is a schedule, not a backup.**

# Restore one file. Until you have, you do not have a backup.

Make every quiet system report zero out of what. Full postmortem and the commands.

sgnk.ai/brilliant ↗

**Sagnik Mitra**

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra